Azentio.ai — API Documentation

Version: 1.0.0

Base URL: `http://localhost:8000`

API Prefix: `/api/v1`

OpenAPI UI: `http://localhost:8000/docs`

ReDoc UI: `http://localhost:8000/redoc`

Table of Contents

1. [Overview](#overview)
2. [Authentication Strategy](#authentication-strategy)
3. [Response Envelope](#response-envelope)
4. [Error Codes Reference](#error-codes-reference)
5. [Roles and Permissions](#roles-and-permissions)
6. [Endpoints](#endpoints)
- [Health](#health)
- [Authentication — Login](#post-apiv1authlogin)
- [Authentication — Refresh](#post-apiv1authrefresh)
- [Authentication — Current User](#get-apiv1authme)
- [Phase 2 Product and Enquiry Endpoints](#phase-2-product-and-enquiry-endpoints)
- [Phase 3 Observation and Consent Endpoints](#phase-3-observation-and-consent-endpoints)
7. [Data Models](#data-models)
8. [Validation Rules](#validation-rules)
9. [HTTP Status Code Conventions](#http-status-code-conventions)
10. [Phase 4 Moderation and Product Workflows](#phase-4-moderation-and-product-workflows)

Overview

Azentio.ai is an Intelligent Collaboration Portal. This document covers all endpoints implemented in **Phase 1** of the backend:

Method	Path	Auth Required	Description
GET	`/api/v1/health`	No	Liveness check
POST	`/api/v1/auth/login`	No	Obtain access and refresh tokens
POST	`/api/v1/auth/refresh`	No	Exchange a refresh token for new tokens
GET	`/api/v1/auth/me`	Yes — Bearer token	Return the authenticated user and their memberships

---

Authentication Strategy

Scheme: JWT Bearer tokens (RFC 7519)

Library: **`python-jose`** with HMAC-SHA256 (**`HS256`**)

Password hashing: Argon2 via **`passlib`**

Token types

Token	Lifetime	Claim **`type`**	Purpose
Access token	30 minutes (configurable via **`ACCESS_TOKEN_EXPIRE_MINUTES`**)	**`"access"`**	Authenticate API requests
Refresh token	7 days (configurable via **`REFRESH_TOKEN_EXPIRE_DAYS`**)	**`"refresh"`**	Obtain a new access token without re-entering credentials

JWT payload claims

Claim	Type	Description
`sub`	string (UUID)	User ID
`iat`	integer (Unix timestamp)	Issued at
`exp`	integer (Unix timestamp)	Expiry
`type`	string	**`"access"`** or **`"refresh"`**

Revocation approach

Token revocation is **stateless** in Phase 1. Tokens are valid until expiry. The **`type`** claim is verified on every request — a refresh token cannot be used as an access token and vice versa. A full revocation/blocklist strategy (e.g. Redis-backed jti blocklist) is planned for a later phase.

How to authenticate

Include the access token in the **`Authorization`** header of every protected request:

```
Authorization: Bearer <access_token>
```

Role derivation

Roles are **never** read from the JWT payload. On every authenticated request, roles are loaded from the **`organisation_memberships`** table using the **`sub`** (user ID) claim. The frontend cannot influence role checks.

Response Envelope

All responses — success and error — use a consistent JSON envelope.

Success

```
{
  "success": true,
  "message": "Human-readable message",
  "data": {},
  "meta": {}
}
```

Field	Type	Description
`success`	boolean	Always **`true`** for success responses
`message`	string	Human-readable description of the result
`data`	object or array	The response payload

Field	Type	Description
meta	object	Pagination metadata or empty object **{}**

Error

```
{
  "success": false,
  "message": "Human-readable error message",
  "error": {
    "code": "ERROR_CODE",
    "details": []
  },
  "meta": {}
}
```

Field	Type	Description
success	boolean	Always **false** for error responses
message	string	Human-readable description of the error
error.code	string	Machine-readable error code
error.details	array	Field-level validation errors (populated on **422** only)
meta	object	Always **{}** for error responses

Pagination meta (future paginated endpoints)

```
{
  "page": 1,
  "page_size": 20,
  "total": 150,
  "total_pages": 8
}
```

Error Codes Reference

Code	HTTP Status	Description
INVALID_CREDENTIALS	401	Email not found or password incorrect
AUTHENTICATION_REQUIRED	401	No token provided or token is malformed
INVALID_TOKEN	401	Refresh token is invalid, expired, or wrong type
FORBIDDEN	403	Authenticated but insufficient role/permission
PRODUCT_NOT_FOUND	404	Product is unknown or not published for public access
ENQUIRY_NOT_FOUND	404	Demo enquiry is unknown
OBSERVATION_NOT_FOUND	404	Observation is unknown or outside the caller's tenants
SLUG_CONFLICT	409	Product slug already exists
INVALID_STATUS_TRANSITION	400	Demo enquiry status transition is not allowed
CONSENT_ALREADY_GRANTED	409	Consent is already active
CONSENT_NOT_ACTIVE	409	Consent cannot be withdrawn because it is not currently granted
VALIDATION_ERROR	422	Request body failed Pydantic validation
INTERNAL_SERVER_ERROR	500	Unhandled server-side exception

Roles and Permissions

Roles are stored in the **`organisation_memberships`** table. A user may hold different roles in different organisations. All role checks are performed server-side.

Role	Value in DB	Description
Client	**`client`**	External client user
Moderator	**`moderator`**	Reviews and approves observations
Product Owner	**`product_owner`**	Manages product improvements
Employee	**`employee`**	Internal Azentio employee
Admin	**`admin`**	Full administrative access

Phase 1 permission matrix

Endpoint	Public	client	moderator	product_owner	employee	admin
`GET /health`	■	■	■	■	■	■
`POST /auth/login`	■	■	■	■	■	■
`POST /auth/refresh`	■	■	■	■	■	■
`GET /auth/me`	■	■	■	■	■	■

Endpoints

Health

`GET /api/v1/health`

Purpose: Liveness check. Confirms the API process is running and reachable.

Authentication: None

Roles required: None

Audited: No

Request

No request body. No path or query parameters.

Headers:

Header	Required	Value
(none required)	—	—

Success response — **`200 OK`**

```
{
  "success": true,
  "message": "Service is healthy",
  "data": {
    "status": "ok"
  },
  "meta": {}
}
```

Error responses

Status	Code	Condition
500	**`INTERNAL_SERVER_ERROR`**	Unhandled exception during startup or routing

Example **`500`** error

```
{
  "success": false,
  "message": "An unexpected error occurred",
  "error": {
    "code": "INTERNAL_SERVER_ERROR",
    "details": []
  },
  "meta": {}
}
```

Authentication — Login

`POST /api/v1/auth/login`

Purpose: Authenticate a user with email and password. Returns a short-lived access token and a long-lived refresh token.

Authentication: None

Roles required: None

Audited: No (login failure auditing planned for a later phase)

Request

Headers:

Header	Required	Value
`Content-Type`	Yes	**`application/json`**

Request body:

Field	Type	Required	Validation
`email`	string	Yes	Valid email format (RFC 5322)
`password`	string	Yes	Must not be blank or whitespace-only

```
{
  "email": "alice.smith@acmecorp.com",
  "password": "SecurePass123!"
}
```

Success response — **`200 OK`**

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMWIyYzNkNC1lNWY2LTc4OTAtYWJjZC1lZjEyMzQ1Njc4OTAiI
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMWIyYzNkNC1lNWY2LTc4OTAtYWJjZC1lZjEyMzQ1Njc4OTAi
    "token_type": "bearer"
  },
  "meta": {}
}
```

`data` fields:

Field	Type	Description
`access_token`	string	JWT access token. Valid for 30 minutes.

Field	Type	Description
`refresh_token`	string	JWT refresh token. Valid for 7 days.
`token_type`	string	Always **`"bearer"`**

Error responses

Status	Code	Condition
401	**`INVALID_CREDENTIALS`**	Email not found, password incorrect, or account inactive
422	**`VALIDATION_ERROR`**	Missing fields, invalid email format, or blank password

`401` — Invalid credentials (generic message — prevents user enumeration):

```
{
  "success": false,
  "message": "Invalid email or password",
  "error": {
    "code": "INVALID_CREDENTIALS",
    "details": []
  },
  "meta": {}
}
```

`422` — Validation error (example: missing password field):

```
{
  "success": false,
  "message": "Request validation failed",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [
      {
        "field": "body.password",
        "msg": "Field required"
      }
    ]
  },
  "meta": {}
}
```

`422` — Validation error (example: invalid email format):

```
{
  "success": false,
  "message": "Request validation failed",
  "error": {
    "code": "VALIDATION_ERROR",
    "details": [
      {
        "field": "body.email",
        "msg": "value is not a valid email address"
      }
    ]
  },
  "meta": {}
}
```

Security notes

• The same **`401 INVALID_CREDENTIALS`** response is returned whether the email does not exist, the password is wrong, or the account is inactive. This prevents user enumeration.

• Passwords are hashed with Argon2. The plaintext password is never stored or logged.

• The response never contains the hashed password or any internal user fields.

Authentication — Refresh

`POST /api/v1/auth/refresh`

Purpose: Exchange a valid refresh token for a new access token and a new refresh token (token rotation).

Authentication: None (the refresh token itself is the credential)

Roles required: None

Audited: No

Request

Headers:

Header	Required	Value
`Content-Type`	Yes	**`application/json`**

Request body:

Field	Type	Required	Validation
`refresh_token`	string	Yes	Must be a valid, non-expired JWT with **`type: "refresh"`**

```
{
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMWIyYzNkNC1lNWY2LTc4OTAtYWJjZC1lZjEyMzQ1Njc4OTAiL
}
```

Success response — **`200 OK`**

```
{
  "success": true,
  "message": "Token refreshed",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMWIyYzNkNC1lNWY2LTc4OTAtYWJjZC1lZjEyMzQ1Njc4OTAiL
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhMWIyYzNkNC1lNWY2LTc4OTAtYWJjZC1lZjEyMzQ1Njc4OTAi
    "token_type": "bearer"
  },
  "meta": {}
}
```

Error responses

Status	Code	Condition
401	**`INVALID_TOKEN`**	Token is expired, malformed, has wrong **`type`** claim, or the user no longer exists / is inactive
422	**`VALIDATION_ERROR`**	**`refresh_token`** field is missing from the request body

`401` — Invalid token:

```
{
  "success": false,
  "message": "Invalid or expired refresh token",
  "error": {
    "code": "INVALID_TOKEN",
    "details": []
  },
  "meta": {}
}
```

Security notes

• Passing an access token to this endpoint returns **`401 INVALID_TOKEN`** because the **`type`** claim is checked.

• The user's **`is_active`** flag is re-verified on every refresh. A deactivated user cannot obtain new tokens.

• Both a new access token and a new refresh token are issued on each call (token rotation).

Authentication — Current User

`GET /api/v1/auth/me`

Purpose: Return the profile and organisation memberships of the currently authenticated user.

Authentication: Required — Bearer access token

Roles required: Any authenticated user with an active account

Audited: No

Request

No request body. No path or query parameters.

Headers:

Header	Required	Value
`Authorization`	Yes	**`Bearer`**

```
GET /api/v1/auth/me HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Success response — **`200 OK`**

```
{
  "success": true,
  "message": "Current user retrieved",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "email": "alice.smith@acmecorp.com",
    "full_name": "Alice Smith",
    "is_active": true,
    "is_verified": true,
    "created_at": "2024-11-01T09:00:00Z",
    "memberships": [
      {
        "organisation_id": "f1e2d3c4-b5a6-7890-fedc-ba0987654321",
        "organisation_name": "Acme Corp",
        "role": "client",
        "is_active": true
      }
    ]
  },
  "meta": {}
}
```

`data` fields:

Field	Type	Description
`id`	UUID string	Unique user identifier
`email`	string	User's email address
`full_name`	string	User's display name
`is_active`	boolean	Whether the account is active
`is_verified`	boolean	Whether the email address has been verified
`created_at`	ISO 8601 datetime	Account creation timestamp (UTC)
`memberships`	array	List of organisation memberships

`memberships[]` fields:

Field	Type	Description
`organisation_id`	UUID string	Organisation identifier
`organisation_name`	string	Organisation display name
`role`	string	Role within this organisation. One of: **`client`**, **`moderator`**, **`product_owner`**, **`employee`**, **`admin`**
`is_active`	boolean	Whether this membership is currently active

Error responses

Status	Code	Condition
401	**`AUTHENTICATION_REQUIRED`**	No **`Authorization`** header, token is malformed, expired, or has wrong **`type`** claim
401	**`AUTHENTICATION_REQUIRED`**	User referenced by token no longer exists or is inactive

`401` — Authentication required:

```
{
  "success": false,
  "message": "Authentication required",
  "error": {
    "code": "AUTHENTICATION_REQUIRED",
    "details": []
  },
  "meta": {}
}
```

Security notes

• The **`hashed_password`** field is never included in any response.

• Roles are loaded from the database on every request — they are not read from the JWT payload.

• Only memberships belonging to the authenticated user are returned. There is no way to request another user's profile via this endpoint.

• An access token used as a refresh token (or vice versa) is rejected because the **`type`** claim is validated.

Data Models

User

Stored in the **`users`** table.

Column	Type	Constraints	Description
`id`	UUID	PK, not null	Auto-generated UUID v4
`email`	VARCHAR(255)	Unique, not null, indexed	Login identifier
`full_name`	VARCHAR(255)	Not null	Display name
`hashed_password`	VARCHAR(255)	Not null	Argon2 hash — never exposed via API
`is_active`	BOOLEAN	Not null, default **`true`**	Soft-disable without deletion
`is_verified`	BOOLEAN	Not null, default **`false`**	Email verification flag
`created_at`	TIMESTAMPTZ	Not null	Set on insert
`updated_at`	TIMESTAMPTZ	Not null	Updated on every write

Organisation

Stored in the **`organisations`** table.

Column	Type	Constraints	Description
`id`	UUID	PK, not null	Auto-generated UUID v4
`name`	VARCHAR(255)	Not null	Display name
`slug`	VARCHAR(100)	Unique, not null, indexed	URL-safe identifier
`description`	TEXT	Nullable	Optional description
`is_active`	BOOLEAN	Not null, default **`true`**	Soft-disable
`created_at`	TIMESTAMPTZ	Not null	Set on insert

OrganisationMembership

Stored in the **`organisation_memberships`** table. Unique constraint on **`(user_id, organisation_id)`**.

Column	Type	Constraints	Description
`id`	UUID	PK, not null	Auto-generated UUID v4
`user_id`	UUID	FK → users.id (CASCADE), indexed	Member user
`organisation_id`	UUID	FK → organisations.id (CASCADE), indexed	Member organisation
`role`	ENUM	Not null	One of **`client`**, **`moderator`**, **`product_owner`**, **`employee`**, **`admin`**
`is_active`	BOOLEAN	Not null, default **`true`**	Membership active flag
`joined_at`	TIMESTAMPTZ	Not null	Set on insert

Product

Stored in the **`products`** table. Public and staff routes are implemented in Phase 2.

Column	Type	Constraints	Description
`id`	UUID	PK, not null	Auto-generated UUID v4
`name`	VARCHAR(255)	Not null	Product display name
`slug`	VARCHAR(100)	Unique, not null, indexed	URL-safe identifier
`description`	TEXT	Nullable	Optional description
`category`	VARCHAR(100)	Nullable, indexed	Product category
`is_published`	BOOLEAN	Not null, default **`false`**	Controls public visibility
`created_at`	TIMESTAMPTZ	Not null	Set on insert
`updated_at`	TIMESTAMPTZ	Not null	Updated on every write

Observation

Stored in the **`observations`** table. Private by default and scoped to one organisation.

Column	Type	Constraints	Description
`id`	UUID	PK, not null	Observation identifier
`title`	VARCHAR(255)	Not null, indexed	Observation title
`description`	TEXT	Not null	Observation details
`product_id`	UUID	FK → products.id (SET NULL), nullable, indexed	Optional associated product

Column	Type	Constraints	Description
category	VARCHAR(100)	Nullable, indexed	Optional category
created_by_user_id	UUID	FK → users.id (CASCADE), not null, indexed	Creator
organisation_id	UUID	FK → organisations.id (CASCADE), not null, indexed	Owning organisation
status	ENUM	Not null	**private** or **in_review**
consent_status	ENUM	Not null, indexed	**not_granted**, **granted**, or **withdrawn**
Grant and withdrawal fields	TIMESTAMP/UUID	Nullable	Current consent actor and timestamp
created_at / **updated_at**	TIMESTAMP	Not null	Resource timestamps

ObservationConsentEvent

Stored in **observation_consent_events**. Each grant or withdrawal is appended as an audit event.

Column	Type	Constraints	Description
id	UUID	PK, not null	Event identifier
observation_id	UUID	FK → observations.id (CASCADE), indexed	Related observation
action	ENUM	Not null, indexed	Consent action (**granted** or **withdrawn**)
actor_user_id	UUID	FK → users.id (CASCADE), indexed	User who changed consent
organisation_id	UUID	FK → organisations.id (CASCADE), indexed	Organisation ownership at time of action
created_at	TIMESTAMP	Not null	Action timestamp
reason	VARCHAR(500)	Nullable	Optional audit reason

Phase 2 Product and Enquiry Endpoints

Method	Path	Authentication	Permission
GET	**/products**	None	Public; published products only
GET	**/products/{product_id}**	None	Public; published products only
POST	**/products**	Bearer JWT	**admin** or **employee**
PATCH	**/products/{product_id}**	Bearer JWT	**admin** or **employee**
DELETE	**/products/{product_id}**	Bearer JWT	**admin** or **employee**
POST	**/demo-enquiries**	None	Public submission
GET	**/demo-enquiries**	Bearer JWT	**admin**, **employee**, or **moderator**
GET	**/demo-enquiries/{enquiry_id}**	Bearer JWT	**admin**, **employee**, or **moderator**
PATCH	**/demo-enquiries/{enquiry_id}/status**	Bearer JWT	**admin**, **employee**, or **moderator**

Key product rules: explicit slug conflicts return **409 SLUG_CONFLICT**; deactivation changes **is_published** to false and removes the product from public detail/list routes. Demo enquiry status transitions are **new -> in_review -> responded -> closed**, with closing permitted from active states.

See [Phase 2 API documentation](phase2_api_documentation.md) for schemas, examples, statuses, and limitations.

Phase 3 Observation and Consent Endpoints

Method	Path	Authentication	Permission
POST	**`/observations`**	Bearer JWT	Active **`client`** membership in submitted organisation
GET	**`/observations`**	Bearer JWT	Active **`client`**; optional organisation filter
GET	**`/observations/{observation_id}`**	Bearer JWT	Active **`client`** membership in owning organisation
POST	**`/observations/{observation_id}/consent`**	Bearer JWT	Active **`client`** membership in owning organisation
DELETE	**`/observations/{observation_id}/consent`**	Bearer JWT	Active **`client`** membership in owning organisation

Observations are private on creation. Consent is not automatic. Grant and withdrawal both require a server-side active **`client`** membership in the observation organisation, and each action is persisted as a timestamped audit event. Withdrawal sets consent to **`withdrawn`**; future sharing logic must accept only **`granted`** observations.

See [Phase 3 API documentation](phase3_api_documentation.md) for request/response examples, errors, and schema details.

Validation Rules

Endpoint	Field	Rule
`POST /auth/login`	**`email`**	Required. Must be a valid email address (RFC 5322). Validated by Pydantic **`EmailStr`**.
`POST /auth/login`	**`password`**	Required. Must not be blank or whitespace-only.
`POST /auth/refresh`	**`refresh_token`**	Required. Must be a non-empty string. JWT validity is checked at the service layer.
All endpoints	Request body	Extra fields are ignored. Missing required fields return **`422 VALIDATION_ERROR`**.

HTTP Status Code Conventions

Status	Meaning	When used
200	OK	Successful GET, POST (non-creation), PATCH, and DELETE
201	Created	Successful resource creation
400	Bad Request	Semantic errors such as invalid status transitions
401	Unauthorized	Missing, invalid, or expired authentication credential
403	Forbidden	Authenticated but lacking required role or permission
404	Not Found	Resource does not exist or is not visible to the caller
422	Unprocessable Entity	Pydantic request body validation failure
500	Internal Server Error	Unhandled server exception

Phase 4 Moderation and Product Workflows

Phase 4 endpoints are documented in full in [phase4_api_documentation.md](./phase4_api_documentation.md).

Method	Path	Role
GET	**`/api/v1/moderation/observations`**	Moderator, administrator
POST	**`/api/v1/moderation/observations/{observation_id}/decision`**	Moderator, administrator
POST	**`/api/v1/moderation/observations/{observation_id}/publish`**	Moderator, administrator
POST	**`/api/v1/entitlements/product`**	Administrator
GET	**`/api/v1/shared-learning`**	Active client; other roles are denied
GET	**`/api/v1/shared-learning/{entry_id}`**	Active client
DELETE	**`/api/v1/shared-learning/{entry_id}`**	Moderator, administrator
POST	**`/api/v1/product-improvements`**	Administrator, product owner
GET	**`/api/v1/product-improvements`**	Authenticated, visibility restricted by role and organisation
PATCH	**`/api/v1/product-improvements/{improvement_id}/status`**	Administrator, product owner
GET	**`/api/v1/audit-events`**	Moderator, administrator

Moderation decisions require redacted content for approvals. Published entries become visible only to entitled client organisations and remain visible only while the source observation consent is granted. Consent withdrawal and manual unpublication both invalidate shared access, and audit events are append-only.

Generated from source and migration inspection. Includes Phase 1, Phase 2, Phase 3, and Phase 4 implementation status.